

NimbusDB Administration Guide

Version 4.2 | Last updated: March 2025 | Internal Engineering Documentation

1. Overview

NimbusDB is a distributed, column-oriented database designed for high-throughput analytical workloads. This guide covers installation, cluster configuration, backup procedures, and troubleshooting for administrators responsible for production deployments. It assumes familiarity with Linux system administration and basic networking concepts.

1.1 Supported Platforms

NimbusDB officially supports Ubuntu 20.04+, RHEL 8+, and Debian 11+. Docker and Kubernetes deployments are supported via the official Helm chart. macOS is supported for local development only and is NOT recommended for production use.

2. Installation

Download the release tarball matching your platform from the internal artifact repository. The following commands install NimbusDB on a standard Ubuntu host:

```
curl -O https://artifacts.internal/nimbusdb/nimbusdb-4.2.0-linux-amd64.tar.gz
tar -xzf nimbusdb-4.2.0-linux-amd64.tar.gz
sudo ./install.sh --data-dir /var/lib/nimbusdb --config /etc/nimbusdb/nimbus.conf
```

Note: The installer requires root privileges to create the nimbusdb system user and register the systemd service. If you are installing in a container, use --no-systemd to skip this step.

2.1 Minimum Hardware Requirements

Deployment Size	CPU Cores	Memory	Disk (SSD)	Network
Development	2	4 GB	20 GB	1 Gbps
Small production	8	32 GB	500 GB	10 Gbps
Large production	32+	128 GB+	4 TB+	25 Gbps

3. Cluster Configuration

A NimbusDB cluster consists of one or more coordinator nodes and a pool of worker nodes. Coordinators handle query planning and metadata; workers store and process data shards. For fault tolerance, run an odd number of coordinators (3 or 5) to maintain quorum.

3.1 Key Configuration Parameters

- cluster.replication_factor — number of copies of each shard (default: 3)

- `cluster.quorum_reads` — require majority acknowledgment on reads (default: false)
- `storage.compaction_interval` — background compaction frequency (default: 6h)
- `network.heartbeat_timeout_ms` — node failure detection threshold (default: 5000)

4. Backup and Recovery

Nightly snapshots are taken automatically when `backup.enabled` is set to `true`. Snapshots are written to the configured object store target and retained according to `backup.retention_days`. Point-in-time recovery is supported for up to 30 days by replaying the write-ahead log against the nearest snapshot.

WARNING: Restoring a snapshot to a running cluster will overwrite existing data on the target shards. Always restore to a scratch cluster first and verify data integrity before promoting.

4.1 Manual Backup Procedure

- 1 Put the target coordinator into maintenance mode: `nimbusctl maintenance enable`
- 2 Trigger a manual snapshot: `nimbusctl backup create --label pre-migration`
- 3 Verify the snapshot manifest was written to the object store
- 4 Disable maintenance mode: `nimbusctl maintenance disable`

5. Troubleshooting

Common issues and resolutions are summarized below.

Symptom	Likely Cause	Resolution
Node marked DOWN repeatedly	Network latency spikes	Increase <code>heartbeat_timeout_ms</code>
Slow query planning	Stale statistics	Run <code>nimbusctl analyze <table></code>
Disk usage growing unbounded	Compaction disabled/failing	Check compaction logs, re-enable

For further assistance, contact the Database Reliability team on [#nimbusdb-support](#).